

# From Expert Questions to a Verifiable Search Architecture

A research analysis of Leant2's further-improvement proposals

Research report prepared for the Leant2 project

21 September 2026

Repository snapshot: 784664ff34e819422510a4d470ab07fe48bb736b

Inspected toolchain: Lean v4.34.0

## Abstract

This report answers all 31 questions in the nine research areas of Leant2's *Further Improvements* proposal. The central recommendation is to separate kernel-certified acceptance, solution-preserving pruning, and reproducible search and ranking: none of these guarantees implies the other two. Concrete answers include dependency-separated search over persistent Lean states; bounded joint synthesis of carriers, algebras, and finishers; residual observation graphs with rollback-safe invalidation; native motive construction; proof-producing evaluation; and dictionary-aware library retrieval.

Several premises of the proposal require correction. Reverse is an ordinary right fold into lists, although an endomorphism carrier can improve its cost. Finite observations do not determine the minimal carrier of an unknown function, and partial observation rows cannot simply be counted as states. Counting open goals is not generally an admissible search heuristic. Smyth already supports higher-order examples in its implementation, Hoogle+ already handles type classes, and Lean 4.34 provides `cbv` and `decide_cbv` for proof-producing evaluation. The report gives explicit correctness arguments, bounded completeness statements, implementation interfaces, regression tests, and an evaluation plan. Included Python experiments check small semantic claims; no Lean compilation, Leant2 benchmark rerun, or model accuracy experiment is claimed.

**Reading guide.** Sections 2–10 follow the proposal's R1–R9 numbering. Every question has its own identifier, such as R2.3. Sections 11 and 12 combine the answers into an implementation sequence. Appendix A is a complete question index; Appendix B records the executable checks.

## Contents

|          |                                                                                     |           |
|----------|-------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Scope, evidence, and the guarantees that matter</b>                              | <b>4</b>  |
| 1.1      | What was inspected . . . . .                                                        | 4         |
| 1.2      | Three independent guarantees . . . . .                                              | 4         |
| 1.3      | Seven immediate corrections to the proposal . . . . .                               | 5         |
| 1.4      | Decision overview . . . . .                                                         | 5         |
| <b>2</b> | <b>R1: search with coupled goals</b>                                                | <b>6</b>  |
| 2.1      | R1.1: can shared goals be organized without copying everything? . . . . .           | 6         |
| 2.2      | R1.2: what heuristic is genuinely admissible? . . . . .                             | 6         |
| 2.3      | R1.3: can online cost learning coexist with defensible ranking? . . . . .           | 7         |
| 2.4      | R1.4: can wall-clock results be independent of machine speed? . . . . .             | 8         |
| <b>3</b> | <b>R2: carriers, accumulators, and invented state</b>                               | <b>8</b>  |
| 3.1      | R2.1: can examples determine a minimal carrier type? . . . . .                      | 8         |
| 3.2      | R2.2: what is the alphabet for a polymorphic function? . . . . .                    | 9         |
| 3.3      | R2.3: can fusion be run backwards, and what can automata prove? . . . . .           | 10        |
| 3.4      | R2.4: can motive enumeration cover primitive recursion efficiently? . . . . .       | 12        |
| 3.5      | R2.5: do generalization and lemma-discovery heuristics transfer? . . . . .          | 12        |
| <b>4</b> | <b>R3: evaluation of partial programs</b>                                           | <b>13</b> |
| 4.1      | R3.1: what is already known about higher-order and dependent examples? . . .        | 13        |
| 4.2      | R3.2: how should an assignment-driven normalizer survive backtracking? . . .        | 14        |
| 4.3      | R3.3: what is the cheapest sound fast-evaluation path? . . . . .                    | 14        |
| 4.4      | R3.4: does top-down angelic recursion admit correctness guarantees? . . . . .       | 15        |
| <b>5</b> | <b>R4: dependent motives, generalization, and transport</b>                         | <b>16</b> |
| 5.1      | R4.1: can a useful motive class be enumerated completely? . . . . .                 | 16        |
| 5.2      | R4.2: how can transports be handled without destroying identity? . . . . .          | 17        |
| 5.3      | R4.3: what unfolding policy should search use? . . . . .                            | 17        |
| 5.4      | R4.4: is native motive computation available as a library? . . . . .                | 18        |
| <b>6</b> | <b>R5: recursive bodies and checked realization</b>                                 | <b>19</b> |
| 6.1      | R5.1: does a capability interpreter agree with the realized definition? . . . . .   | 19        |
| 6.2      | R5.2: what changes when angelic execution is top-down? . . . . .                    | 20        |
| 6.3      | R5.3: is there a small canonical recursion basis with measured coverage? . . . . .  | 20        |
| <b>7</b> | <b>R6: contracts, induction, and proof debt</b>                                     | <b>21</b> |
| 7.1      | R6.1: how should value holes and proof holes be interleaved? . . . . .              | 21        |
| 7.2      | R6.2: which properties have a recognizable induction-along-the-program proof? . . . | 21        |
| 7.3      | R6.3: what Lean induction automation is usable now? . . . . .                       | 22        |

|           |                                                                                |           |
|-----------|--------------------------------------------------------------------------------|-----------|
| 7.4       | R6.4: is there a principled policy for proof debt? . . . . .                   | 23        |
| <b>8</b>  | <b>R7: library retrieval with dependency and type classes</b>                  | <b>24</b> |
| 8.1       | R7.1: what abstraction is sound and useful for dependent signatures? . . . . . | 24        |
| 8.2       | R7.2: has type-guided refinement already handled type classes? . . . . .       | 24        |
| 8.3       | R7.3: can premise selection consume data goals and examples? . . . . .         | 25        |
| <b>9</b>  | <b>R8: ranking, equivalence, and ambiguity</b>                                 | <b>26</b> |
| 9.1       | R8.1: what formal objective should rank inhabitants? . . . . .                 | 26        |
| 9.2       | R8.2: which candidate equivalence is cheap and meaningful? . . . . .           | 27        |
| <b>10</b> | <b>R9: learned proposals without expanding the trust boundary</b>              | <b>27</b> |
| 10.1      | R9.1: what accuracy can be claimed for model-proposed carriers? . . . . .      | 27        |
| 10.2      | R9.2: what makes learned guidance deterministic and auditable? . . . . .       | 28        |
| <b>11</b> | <b>An integrated Lean-native architecture</b>                                  | <b>29</b> |
| 11.1      | Keep authority at the original query . . . . .                                 | 29        |
| 11.2      | Proposed interfaces . . . . .                                                  | 29        |
| 11.3      | A deterministic control loop . . . . .                                         | 30        |
| 11.4      | Negative-result algebra . . . . .                                              | 30        |
| <b>12</b> | <b>Implementation sequence and falsifiable evaluation</b>                      | <b>31</b> |
| 12.1      | A dependency-aware sequence . . . . .                                          | 31        |
| 12.2      | The regression matrix . . . . .                                                | 31        |
| 12.3      | Metrics and ablations . . . . .                                                | 32        |
| 12.4      | What remains genuinely empirical . . . . .                                     | 32        |
| <b>13</b> | <b>Conclusion</b>                                                              | <b>33</b> |
| <b>A</b>  | <b>Complete question-to-answer index</b>                                       | <b>33</b> |
| <b>B</b>  | <b>Executable semantic checks and their limits</b>                             | <b>34</b> |
| <b>C</b>  | <b>Evidence status and reproducibility</b>                                     | <b>35</b> |

# 1 Scope, evidence, and the guarantees that matter

## 1.1 What was inspected

The requested proposal is the directory docs/proposals/11-further-improvements in the pinned repository snapshot [1]. Its question-bearing files are 05-search.tex through 12-ranking.tex; the last contains both R8 and R9. The expert-contact summary in 15-experts.tex groups those questions rather than introducing another independent list. There are  $4 + 5 + 4 + 4 + 3 + 4 + 3 + 2 + 2 = 31$  questions.

The analysis also inspected the acceptance gate, transactional state, and engine implementation [2]. This matters because a design recommendation must fit the actual Lean-native architecture, not a generic synthesis system. The current engine uses metavariable-state snapshots, iterative deepening, wall-clock lane deadlines, and a short grace period after the first accepted candidate. Its ranking uses unused binders, eliminator counts, and expression size, with printed-form deduplication. The acceptance path kernel-checks declarations synchronously and audits their transitive axiom dependencies.

The proposal’s performance tables are *reported measurements*, not new experiments in this report. In particular, its acceptance table is explicitly anchored to the earlier commit 33cec8b, not to the snapshot inspected here. Its approximately 45% residual-evaluation share and approximately 1% state-save/restore share come from one contract-heavy query; they do not prove that large best-first frontiers have negligible memory cost [3]. The distinction is important when prioritizing a rewrite.

## 1.2 Three independent guarantees

Let a frozen query be  $Q = (\Gamma, T, C, \mathcal{A}, \mathcal{G})$ , where  $T$  is the requested type,  $C$  is an optional contract,  $\mathcal{A}$  is the permitted axiom profile, and  $\mathcal{G}$  records the selected search grammar and bounds. The following are different claims.

**Acceptance correctness.** An accepted result contains a term  $p$  and, when required, a proof  $\pi$ , with

$$\Gamma \vdash p : T, \quad \Gamma \vdash \pi : C(p), \quad \text{axioms}(p, \pi) \subseteq \mathcal{A}.$$

The kernel checks the judgments; a request-binding layer must ensure that these are the *requested* judgments. A kernel cannot detect that a caller silently substituted a different target or contract.

**Pruning correctness.** For a partial program  $p$  with a dependent telescope of holes  $\Delta$ , a hard refutation must establish that no permitted completion can satisfy the query:

$$\forall \theta : \Delta, \quad B(\theta) \longrightarrow \neg C(p[\theta]). \quad (1)$$

Here  $B$  records the branch’s genuine assumptions, not speculative model predictions. A final kernel gate does not repair the loss of a correct program that was wrongly pruned. Conversely, an evaluator may be very useful while remaining incomplete: it can return *unknown* without endangering soundness.

**Search and publication reproducibility.** A fixed logical-work budget, grammar, environment, proposal manifest, and scheduler should determine the same search trace and published candidate order. A fixed wall-clock deadline generally determines only a machine-dependent prefix of that trace. Kernel correctness says nothing about which valid result appears first.

These distinctions should appear in data types and receipts, not merely in comments. A result may be certified but not optimal; a branch may be unresolved rather than refuted; and a bounded grammar may be exhausted without proving that the original Lean type is uninhabited.

### 1.3 Seven immediate corrections to the proposal

The detailed arguments below imply seven changes before implementing the larger research plan.

First, replace the claim that one unit per open non-leaf goal is an admissible heuristic with a proved lower bound or zero. Shared unification can discharge several goals at once. Second, distinguish minimal *semantic state*, minimal *type expression*, and minimal or efficient *program*; these are different carrier objectives. Third, do not use reverse as evidence that a list-valued right fold is impossible. Fourth, require an actual upper bound on reachable carrier states before using a cardinality refutation; counting values found so far supplies the opposite kind of bound.

Fifth,  $n + 0$  is definitionally equal to  $n$  in Lean; an example requiring transport should use  $0 + n$  or another genuinely non-definitional equality [16]. Sixth, observational coincidence is a useful display relation but is not generally a congruence for higher-order or recursive synthesis. Seventh, update the related-work picture: Smyth, Hoogse, and Lean’s current evaluator already answer important parts of R3, R5, and R7 [13, 23, 15].

### 1.4 Decision overview

| Area | Recommended first implementation                              | What must not be claimed                                                           |
|------|---------------------------------------------------------------|------------------------------------------------------------------------------------|
| R1   | Persistent whole-state queue with dependency components       | Independent goals from target syntax alone; machine-independent wall-clock results |
| R2   | Bounded joint carrier–algebra–finisher synthesis              | A globally minimal intended carrier learned from finite examples                   |
| R3   | Observation DAG plus proof-producing refutation backend       | Differential tests as a proof of conservativity                                    |
| R4   | Native induction adapter and dependency-closed generalization | Complete general motive inference; arbitrary cast erasure                          |
| R5   | Typed recursive capabilities with checked realization         | Automatic definitional equality between two interpreters                           |
| R6   | Local invariant obligations and watcher-driven proof debt     | A complete syntactic recognizer for inductively provable properties                |
| R7   | Typed hypergraph retrieval with explicit dictionaries         | Sound negative reachability for rules omitted from the abstraction                 |
| R8   | Frozen publication objective and layered equivalence          | Sample equivalence as unrestricted semantic equality                               |
| R9   | Optional recorded proposals with deterministic replay         | Temperature zero as a reproducibility guarantee; unmeasured accuracy               |

## 2 R1: search with coupled goals

### 2.1 R1.1: can shared goals be organized without copying everything?

**Answer.** Yes: use a persistent *whole-state OR graph*, and factor only certified independent parts of a state into AND components. This does not eliminate alternative assignments to genuinely shared variables. It avoids treating all goals as either globally independent or globally sequential.

A state should contain the exact Lean saved state, the pending goals, delayed constraints, residual observations, immutable provenance, and the costs needed by the scheduler. Build a dependency graph whose vertices include expression metavariables, universe metavariables, and pending constraints. A goal's support is the transitive closure through its target, local declaration types and values, assignments, and delayed dependencies. A variable absent from the printed target can still be reachable through a local's type. A shared universe constraint can couple apparently unrelated data goals.

For two candidate components, compute or conservatively approximate their read and write footprints. If either can assign a variable that the other reads, they belong in the same component. Unknown tactic effects also prevent separation. Shared *fixed* locals need not connect components; shared revisable state does. A small separator of shared variables can be assigned first, after which the remaining components often become independent.

**Proposition 2.1** (Conditional factorization). *Suppose a state's obligations partition into components with disjoint mutable sets  $W_1, \dots, W_k$ . Every transition for component  $i$  reads only immutable shared data and  $W_i$ , writes only  $W_i$  (including fresh variables allocated exclusively to that component), and leaves all cross-component constraints unchanged. Then transitions in different components commute, and successful component solutions can be combined in any order.*

*Proof.* A transition on  $W_i$  cannot change the inputs observed by a transition on  $W_j$  for  $i \neq j$ , so the two compositions have the same result on each mutable set and on shared data. Repeatedly exchange adjacent independent transitions to group a successful interleaving by component. Conversely, concatenating successful component traces preserves every component's preconditions. The assumptions ensure no omitted joint obligation remains.  $\square$

This theorem is an interface contract, not a claim that arbitrary Lean tactics already satisfy it. Start with conservative footprints for native synthesis rules; treat a general tactic as potentially global unless sandboxed.

**Research evidence.** Aesop provides the relevant practical comparison: its treatment of shared metavariables includes copying and reapplying affected goals rather than assuming independent subproofs. Canonical-min provides a more recent precedent for postponing constraints on blockers and resuming them under assignments in persistent, backtracking search. Its own calculus is not a replacement for Lean's authoritative expressions [4, 6].

**Implementation consequence.** Memoize a solved component as a telescope-abstracted, typed term template, with its environment and assumption fingerprint. Instantiate it with fresh universe and expression variables, substitute actual locals, and recheck its type. Never move raw free-variable identifiers between unrelated contexts. Cache failure only with a bounded-exhaustion justification or a checked refutation, not because an earlier proof attempt timed out.

### 2.2 R1.2: what heuristic is genuinely admissible?

**Answer.** The universally safe starting point is  $h(s) = 0$ . Useful nonzero bounds must be lower bounds on *joint completion cost*, not counts of obligations.

Consider two obligations that become reflexive after the same metavariable assignment. If that assignment has cost one and reflexivity has cost zero, the remaining cost is one, although there are two open obligations. The same phenomenon occurs when a shared type or index choice makes several exact-local steps possible. Merely requiring goals to be “non-leaf” before the assignment does not prevent the counterexample.

Let  $L_i(s)$  be an admissible lower bound obtained by relaxing all other obligations while keeping goal  $i$ . Then

$$h_{\max}(s) = \max_i L_i(s)$$

is admissible whenever the relaxed cost is no greater than the corresponding joint cost. Summing the  $L_i$  is justified only when costs have been partitioned into non-overlapping resources, for example independent components satisfying Proposition 2.1. A more informative relaxation can retain a small separator and compute a joint minimum over its finite abstract assignments.

There are two additional traps. First, rule-application cost is not necessarily final-expression size: filling a shared hole can duplicate its term in the fully expanded expression, while simplification can erase work. A bound for one objective is not automatically a bound for the other. Second, zero-cost rules can create infinitely many states of equal priority. Saturate a terminating zero-cost normalization phase, or detect cycles in the zero-cost state graph, before claiming that cost-ordered enumeration is fair.

**Implementation consequence.** Begin with uniform-cost search under nonnegative integral costs and a measured normalization phase. Add a heuristic only together with a small counterexample suite and a statement of its relaxation. Maintain a separate heuristic field for useful, unproved estimates. An estimate of likely subtree size can guide exploration without being advertised as an A\* lower bound.

### 2.3 R1.3: can online cost learning coexist with defensible ranking?

**Answer.** Yes, if exploration priority and publication rank are separate. Online learning may change what is discovered within a budget. That is a search policy effect, not a logical soundness failure, and it should be visible in a replayable record.

Freeze a publication objective  $J_Q(p)$  when the query is created. It might be a description-length cost plus explicit complexity penalties, as developed in R8.1. The scheduler may learn a different priority

$$P_t(s) = g(s) + \widehat{h}_t(s) - \lambda \widehat{u}_t(s),$$

where  $\widehat{u}_t$  estimates useful progress on observations. This expression is a heuristic, not a lower bound unless independently justified. Learning updates should occur at deterministic logical checkpoints, with stable tie breaking and the same recorded observations.

**Research evidence.** Probe establishes that partial solutions can supply useful just-in-time guidance for bottom-up enumeration. It does not establish that adaptive priorities preserve a particular user’s preferred answer under a fixed time budget [7]. The architecture proposed here uses that guidance while making the publication objective explicit.

Changing edge costs of an existing A\* frontier invalidates its old optimality argument. Either rebuild and consistently reprice the frontier under a fixed new objective, or stop making an optimality claim and keep the learned score solely as exploration guidance. A fair, deterministic baseline lane is useful: it reserves progress for alternatives that a model currently undervalues.

Measure first certified result, best published rank at fixed work, and held-out behavioral quality separately. Rewarding a candidate merely for passing more training examples can favor overfitting. A change in result order should be attributable to the frozen objective or to a recorded search-prefix change, not to timing accidents in an asynchronous model response.

## 2.4 R1.4: can wall-clock results be independent of machine speed?

**Answer.** Not unconditionally while also publishing every improvement reached before the deadline. The appropriate guarantee is deterministic logical prefixes, with explicit truncation at wall-clock cancellation.

**Proposition 2.2** (Deadline incompatibility). *Suppose a deterministic search trace discovers a better candidate after some finite positive amount of additional work. If one machine reaches that point before a fixed deadline and another does not, an anytime interface that publishes its best discovered candidate returns different answers.*

*Proof.* The faster execution has discovered the improvement and must publish it under the stated interface; the slower execution has not. Returning the same answer requires suppressing the improvement or obtaining work beyond the slower execution's completed prefix. Neither is the stated interface.  $\square$

Startup calibration does not remove the issue: node costs vary with reduction, allocation, instance synthesis, and proof search. Counting nodes can improve comparability without accurately predicting time. A logical budget should therefore include the expensive operations or use an explicitly defined weighted ledger, while a wall-clock deadline remains a safety cancellation.

Publish a receipt containing the last completed logical checkpoint, exhausted cost bands, uncompleted tasks, and the fixed candidate ordering. Parallel tasks can be committed in deterministic batches rather than in completion order. The UI may display earlier provisional results, but the stable checkpoint result should be identified separately. Exact reproducibility across machines then means replaying the same checkpoint and manifest, not pretending that the machines perform the same computation in ten seconds.

The existing external ledger, which is not refunded by state rollback, is a good foundation [2]. Extend it to persistent-frontier work and proof tasks. Exclude timestamps, random scratch declaration names, and memory addresses from canonical state and candidate identifiers.

## 3 R2: carriers, accumulators, and invented state

### 3.1 R2.1: can examples determine a minimal carrier type?

**Answer.** Only after specifying a bounded representation problem. Finite examples can determine a smallest *sample-consistent implementation in a finite candidate language*; they do not identify the globally minimal carrier of an unknown intended function.

Write a list implementation as

$$h(x) = \phi(\text{foldr } \delta \ z \ x), \quad z : K, \quad \delta : A \rightarrow K \rightarrow K, \quad \phi : K \rightarrow T. \quad (2)$$

Four objectives immediately differ: the cardinality of reachable states in  $K$ ; the size of the type expression denoting  $K$ ; the description length of  $(K, z, \delta, \phi)$ ; and the operational cost of the resulting program. An infinite carrier may have a tiny type expression. A tiny carrier type may move all the difficult computation into its finisher.

**Example 3.1** (Why finite data does not identify the intended minimum). For any finite set of list inputs, choose a word  $w$  outside that set. The constant-zero function and the function that returns one exactly on  $w$  agree on all those inputs when the observed outputs are zero. The former has a one-state implementation; the latter cannot have a one-state implementation with

a deterministic finisher because it takes two distinct output values. Both are total computable functions over a finite alphabet. Thus the data does not determine even whether the intended function needs more than one state.

An unrestricted finisher makes carrier existence particularly uninformative: choose  $K = \text{List } A$ ,  $z = []$ ,  $\delta = \text{cons}$ , and  $\phi = h$ . The fold reconstructs its input and the finisher performs the entire task. This is a valid representation, but not an auxiliary-state discovery algorithm.

**The bounded problem to implement.** Specify finite grammars for carrier types, seeds, steps, and finishers; bounds on syntax size, literals, providers, helper definitions, and universes; and a fixed cost

$$J(K, z, \delta, \phi) = w_K|K| + w_z|z| + w_\delta|\delta| + w_\phi|\phi|.$$

If the grammars describe a well-typed, total decidable fragment with an exact validator, enumerate its candidates in nondecreasing cost, pruning only by valid necessary conditions. A minimum satisfying the supplied observations is then meaningful. For unrestricted Lean propositions, a candidate whose proof search times out is *unresolved*, so exhausting the enumerator does not by itself establish nonexistence.

**Theorem 3.2** (Bounded synthesis guarantee). *Let  $\mathcal{P}_b$  be a finite effectively enumerated set of well-typed candidate implementations, let sample validation terminate exactly on every member, and let every pruning predicate preserve all sample-consistent members. Exhaustive cost-ordered search either returns a least-cost sample-consistent member or correctly reports that none exists in  $\mathcal{P}_b$ .*

*Proof.* Finiteness and termination of each validation imply termination of exhaustive search. A valid pruning predicate removes no consistent member. Therefore the first consistent member in cost order is minimal among consistent members; if there is none, all members have been excluded by valid tests.  $\square$

The theorem applies to a finite reified DSL or a bounded collection with a proved complete validator. It is not a theorem that arbitrary bounded Lean metaprogram execution always finishes. A practical receipt should distinguish *enumeration exhausted* from *all enumerated candidates resolved*.

**Reverse is a cost example, not an existence obstruction.** For ordinary lists,

$$\text{reverse} = \text{foldr } (\lambda a \ r. r ++ [a]) \ [].$$

This uses  $K = \text{List } A$ . With linked-list append it repeatedly traverses the accumulated suffix, giving quadratic work. The alternative

$$K = \text{List } A \rightarrow \text{List } A, \quad z = \text{id}, \quad \delta(a, k) = \lambda ys. k(a :: ys), \quad \phi(k) = k([])$$

uses one cons operation per element when its closures are applied. A restriction on allowed step syntax may exclude the first program, but the fold characterization alone cannot do so. Carrier synthesis should therefore optimize the whole implementation and state its operational model.

### 3.2 R2.2: what is the alphabet for a polymorphic function?

**Answer.** Abstract elements should be represented by symbolic tokens and their permitted observations, not by a silently chosen finite concrete type. Parametricity justifies this reduction only for a specified fragment.

For a total relationally parametric function  $h_A : \text{List } A \rightarrow \text{List } A$  that has no element-inspecting operations, renaming elements along a function  $f : A \rightarrow B$  gives the naturality equation

$$\text{map } f(h_A(xs)) = h_B(\text{map } f(xs)).$$

At a fixed input shape, a token model can record which input positions are selected, duplicated, or reordered. It cannot manufacture a new abstract  $A$ -value without a supplied source. Such a model is useful for proposing carriers and rejecting impossible data flow. It still leaves infinitely many list shapes and therefore does not establish an unrestricted finite-state learning theorem.

If the context includes equality, order, hashing, or arbitrary callbacks, the available observations change. An equality dictionary allows branches on an equality pattern among tokens; an order dictionary allows order relations. One then needs a symbolic state model with registers or constraints, rather than merely “one state per position.” A BEq dictionary need not be lawful, and its behavior is part of the actual program semantics. Replacing its Boolean result with propositional equality requires an appropriate lawfulness proof.

**Research evidence.** Testing polymorphic properties has a substantial existing theory, including reductions justified by parametricity [11]. That is evidence for exploiting a formally specified polymorphic fragment, not permission to assume that every Lean term, every typeclass dictionary, and every externally supplied provider satisfies the required relational invariance.

**Implementation consequence.** Give each provider a conservative effect signature: token-preserving, shape-observing, equality-observing, order-observing, or opaque. Use positional and equality-pattern abstractions to *propose* states by default. Enable hard parametricity-based pruning only for a certified provider fragment. For indexed families, retain the index and the telescope in which a token is typed; equal-looking tokens in different fibers are not interchangeable.

### 3.3 R2.3: can fusion be run backwards, and what can automata prove?

**Answer.** There is a constructive derivation from a full specification with suitable evidence, and there are complete bounded finite-state searches. Neither is a general inference procedure from a handful of examples to a Lean carrier type.

**Research evidence.** Gibbons, Hutton, and Altenkirch give the fold/unfold characterization; Weber and Caldwell subsequently study constructive versions, including formalization in Nuprl [8, 9]. Consequently, “constructive fold characterization” is not an entirely unexplored question. Its premises concern the function and supporting structure, rather than merely observed input–output pairs.

**Proposition 3.3** (A constructive sufficient condition for a list fold). *Let  $h : \text{List } A \rightarrow B$ , and suppose a section  $s : B \rightarrow \text{List } A$  satisfies  $h(s(b)) = b$ . Assume cons congruence:*

$$h(x) = h(y) \implies h(a :: x) = h(a :: y).$$

*Then  $h = \text{foldr } \delta \ z$ , where  $z = h([])$  and  $\delta(a, b) = h(a :: s(b))$ .*

*Proof.* The base equation holds by definition. For the inductive step,  $h(s(h(x))) = h(x)$ , hence cons congruence gives  $h(a :: s(h(x))) = h(a :: x)$ . Substituting the induction hypothesis for the folded tail yields the desired equation.  $\square$

This is a genuine construction, but it consumes  $h$ ,  $s$ , and a congruence proof. It does not discover them from samples. For a polynomial functor  $F$ , the analogous construction uses a section and an algebra formed by applying  $F(s)$  before the original constructor and  $h$ ; a corresponding congruence condition and induction principle justify it. This is a useful verified transformation once a symbolic specification is available.

**The precise residual-state model.** For a full function  $h : \text{List } A \rightarrow T$ , define

$$x \sim_h y \iff \forall u : \text{List } A, h(u ++ x) = h(u ++ y). \quad (3)$$

The continuation is a *prefix* because a right fold has already processed the tail. This orientation avoids a common prefix/suffix ambiguity.

**Proposition 3.4** (Minimal semantic observer states). *The quotient by  $\sim_h$  supports a right-fold state machine with seed  $[\ ]$ , transition  $\delta(a, [x]) = [a :: x]$ , and output  $\phi([x]) = h(x)$ . Every reachable deterministic state representation satisfying (2) maps onto this quotient. In particular, a finite carrier needs at least as many reachable states as the quotient has classes.*

*Proof.* If  $x \sim_h y$ , then for every  $u$ ,  $h(u ++ (a :: x)) = h((u ++ [a]) ++ x)$  equals the corresponding expression for  $y$ . Thus the transition is well-defined. The empty continuation makes  $\phi$  well-defined. Induction shows that folding  $x$  produces its class. For an arbitrary correct implementation, equal folded states for  $x, y$  remain equal after processing any prefix  $u$ , hence give equal final outputs; so  $x \sim_h y$ . Mapping a reachable folded state to its residual class is therefore well-defined and surjective.  $\square$

This is a semantic quotient, not necessarily finite, decidable, effectively representable, or expressible by the chosen Lean type grammar. It distinguishes what a state must remember without choosing a practical data structure.

**What finite observations really imply.** Create rows indexed by known tails  $x$  and columns indexed by known prefixes  $u$ . An entry is available only when  $h(u ++ x)$  is observed or otherwise proved. Put an incompatibility edge between two rows when some column has *two known different outputs*. Any clique of size  $r$  certifies at least  $r$  distinct states. In contrast, two different partial rows may be compatible: a row known only to be false in column zero and a row known only to be true in column one can describe the same state. Counting distinct partial rows is unsound as a lower-bound argument.

Even graph coloring is only a necessary constraint until transition consistency is enforced: equal state labels for  $x, y$  require equal labels for  $a :: x, a :: y$  whenever these are represented. A finite-state solver can introduce state variables  $q_x$ , transitions  $d_{a,q}$ , and outputs  $o_q$ , with constraints

$$q_{a::x} = d_{a,q_x}, \quad o_{q_x} = h(x)$$

for available observations, plus equality constraints for shared states. Enumerating all transition/output tables at a fixed finite state bound gives a decidable version of the problem. For the exact finite table language, an exhaustive failure is a valid bounded negative result.

**Research evidence.** Angluin’s learning algorithm uses membership and equivalence queries to learn regular languages [10]. A passive set of five to forty examples is not such an oracle. Observation-table closure and consistency cannot be assumed when the necessary entries are unknown.

A second correction concerns the proposal’s count of constructible values. The number of values *found* at a search depth is normally a lower bound on the available states, not an upper bound. Iterating a short step can also produce values much larger than its own syntax. A state-count refutation is valid for a carrier such as `Bool`, whose cardinality is proved, or for an inductively closed abstract state set with a proved finite upper bound. It is not justified by an incomplete value enumerator.

**Implementation consequence.** Use incompatible observations to generate state requirements, propose `Option`, products, and continuations, then synthesize the algebra and finisher jointly.

Where a full algebraic specification exists, try verified fusion or tupling transformations. Otherwise retain fair bounded enumeration as the completeness reference. The automaton is a constraint generator and, in finite cases, a refuter; it is not a magical type decoder.

### 3.4 R2.4: can motive enumeration cover primitive recursion efficiently?

**Answer.** A finite grammar can be enumerated completely up to its stated bound. There is no reason to expect one small finite bound to cover every useful primitive-recursive program with invented parameters. Efficiency requires staging proposals without confusing their priority with completeness.

Use a dependent carrier grammar  $K(\bar{t})$  rather than only a closed type  $K$ . Its first tiers can be the target family itself, functions from existing parameter types, optional element state, and small products. Every proposal must generate its seed, branch algebra, and finisher obligations at the same time. A carrier that is cheap as a type but impossible to initialize is not a promising partial answer.

Three examples explain useful priorities. An optional maximum state explicitly represents “no element seen” and avoids inventing an abstract element for an empty input. A Fibonacci state  $(a, b)$  initialized to  $(0, 1)$  and updated to  $(b, a + b)$  turns the task into ordinary primitive recursion. A right fold implementing a general left-associated reduction may need a continuation; Option A alone should not be assumed sufficient for an arbitrary non-associative combining operation.

For each total cost band, enumerate all well-scoped type expressions from the finite grammar and all bounded implementations under them. Higher-priority schema proposals may be tried first, but eventually process the entire band. A beam or a fixed top- $k$  carrier list is an effective heuristic, not a complete bounded search unless the omitted alternatives remain scheduled elsewhere.

Primitive recursion can be represented by iterated use of recursors, but this observation alone does not solve the search problem: recursion depth, parameter invention, auxiliary functions, and proof obligations need bounds too. State completeness relative to the actual grammar, including its allowed recursor nesting and parameter structure, rather than relative to all primitive recursion in Lean.

### 3.5 R2.5: do generalization and lemma-discovery heuristics transfer?

**Answer.** They transfer as proposal generators, especially when failure exposes a specific missing invariant or auxiliary statistic. They do not supply a complete or automatically trusted carrier learner.

A failed use of an induction hypothesis often indicates one of three changes: an accumulator has a different value; the desired result contains extra information absent from the recursive result; or a recursive result must be combined through an unavailable lemma or helper. Respectively propose a function-valued motive, a tupled state, or a helper with the residual hole’s typed observations. These are more targeted than enumerating every product and function type at the outset.

**Research evidence.** HipSpec combines theory exploration with inductive proving and uses candidate equations generated and tested before proof attempts [12]. That gives a relevant architecture for conjecture generation and rejection. It does not imply that tested equations are proofs, or that the same strategy already invents arbitrary dependent carriers.

For Leant2, record a failure explanation: the induction hypothesis’s type, the required type, changed parameters, blocked equations, and the smallest residual observations that distinguish failed branches. Generate a small family of candidate invariants or helper types from that

explanation. Test them on known examples, then require native Lean proofs of the needed branch conditions. Rank a proposed generalization by the obligations it actually makes solvable, not just by a superficial resemblance between terms.

An especially useful pattern is strengthening

$$h(xs) \rightsquigarrow (h(xs), g(xs)),$$

where  $g$  supplies the missing statistic. The real obligation is closure: both components on  $a :: xs$  must be computable from  $a$  and the strengthened state on  $xs$ . This closure test exposes the difference between inventing a helpful statistic and merely appending another hard synthesis problem.

**Implementation consequence.** Limit failure-driven invention by a deterministic budget and retain its provenance. Reuse a helper only after its implementation and, when relevant, its specification have been certified. Do not introduce a fresh helper axiom and rely on the final program's type to reveal the unsound shortcut.

## 4 R3: evaluation of partial programs

### 4.1 R3.1: what is already known about higher-order and dependent examples?

**Answer.** Higher-order examples are not an entirely missing extension: Smyth's implementation supports them. Dependent examples, however, introduce a separate typing problem that cannot be solved by simply attaching ordinary input–output pairs to a hole.

**Research evidence.** Smyth's formal core and implementation have different scopes. The core presentation uses first-order examples, while the implementation discussion includes higher-order examples and polymorphism. Its live bidirectional evaluation also addresses interdependent holes and examples that are not trace-complete [13]. Therefore the proposal should not treat all those capabilities as absent from the literature. The same source does not supply a Lean-dependent implementation with Lean's conversion and transport semantics.

For Leant2, represent a hole closure by

$$(m, \Gamma_m, \rho, \tau[\rho], \mathcal{O}),$$

where  $m$  identifies the hole,  $\Gamma_m$  is its declaration telescope,  $\rho$  is the environment substitution,  $\tau[\rho]$  is its instantiated expected type, and  $\mathcal{O}$  contains observations in that environment. The telescope and substitution are not optional metadata. A hole of type  $\Pi n : \mathbb{N}, \text{Vec } A \ n$  is observed in different fibers at arguments zero and one; merging those result constraints as untyped values would be meaningless.

For function-valued results, use application observations such as  $k(v_1) = w_1$  and  $k(v_2) = w_2$ . These are constraints on the function, not a claim that a finite table completely describes it. A Church-encoded input can be handled through the applications it performs on supplied eliminators, provided closures and their environments remain explicit.

Backward propagation must also preserve alternatives. Unevaluating a required output through a conditional may yield two disjunctive possibilities: a true guard with a matching first branch, or a false guard with a matching second branch. Choosing just one possibility is a heuristic restriction. The same issue appears when an inverse operation is non-injective. Hard pruning must consider every possibility in the declared bounded grammar or derive a necessary condition valid for all completions.

**Implementation consequence.** Stage the work. First extract independent observations while preserving proofs relating them to the original contract. Next support typed first-order hole

closures. Then add higher-order application observations. Only after those interfaces are stable add dependent fibers, casts, and generalized motives. At every stage, an unsupported construct returns unknown rather than being interpreted under an approximately matching type.

## 4.2 R3.2: how should an assignment-driven normalizer survive backtracking?

**Answer.** Use a residual computation DAG whose cache entries carry validated read sets and immutable assignment identities. Invalidation must distinguish two sibling assignments even when a local version counter has been rolled back.

**Research evidence.** Canonical-min describes suspended constraint continuations indexed by the metavariables blocking them, with resumption after assignments. It is a direct precedent for the requested control pattern, but not a drop-in memoized normalizer for Lean expressions [6]. Adaption supplies broader precedent for dependency-driven, demand-driven incremental computation [14]. The implementation proposed here combines that pattern with Lean state snapshots and typed residual observations.

A cache key should include the expression or closure identity, its environment substitution, the relevant local telescope, the immutable Lean environment, and the evaluator/transparency policy. Its read set records every consulted metavariable assignment and universe assignment, including dependencies reached through already assigned variables. Store the exact immutable assignment-node identity, or a globally unique generation token, not a counter that is restored with the branch.

For example, branch A may assign a hole to zero with version one and cache that the hole equals one as false. After rollback, branch B may assign the same hole to one, also with version one. Reusing the old result is wrong. A branch-qualified immutable assignment identity prevents this ABA pattern. Merely checking whether the hole is still “assigned” does not.

**Proposition 4.1** (Read-set validation). *For a deterministic residual evaluator, suppose a cache entry records every mutable value read during its computation and all other inputs are immutable. If every recorded read has the same value in a new state, replaying the computation produces the same result.*

*Proof.* Assume replay first diverges at some operation. Earlier control flow and intermediate results coincide. A pure operation on those results cannot cause the first divergence. A mutable read cannot either, by the validated read set. These exhaust the allowed operations, giving a contradiction.  $\square$

The theorem requires a complete read set. Tracking only holes that were unassigned at the final stuck point misses earlier assigned values that selected a branch. Also record dependencies on options, local instances, and any environment extension that changes evaluation or inversion rules.

**Implementation consequence.** Implement per-observation watchers before a full custom normalizer. An assignment enqueues only affected observations, and each observation resumes from a retained residual when its dependencies validate. Share closed subexpressions globally within the frozen environment. Share open residuals across branches only after read-set validation. Keep resource charges outside rollback; a restored branch must not receive its spent evaluation budget back.

## 4.3 R3.3: what is the cheapest sound fast-evaluation path?

**Answer.** Prefer an existing proof-producing evaluator and a narrow residual layer before reimplementing the kernel. Also distinguish agreement with kernel *conversion* from agreement

with the program’s *proved equations*. The second is sufficient for certified refutation and can be substantially more practical for well-founded recursion.

**Research evidence.** Lean 4.34 documents `cbv` as call-by-value evaluation that uses function and matcher equations and constructs propositional equality proofs. `decide_cbv` applies it to decision procedures. Unlike native compiled decision, this route does not trust the code generator; the documented proof dependencies are standard Lean axioms, so the actual result must still be audited against the selected profile. The same documentation explicitly notes limitations on rewriting in dependent positions [15]. Lean4Lean’s 2026 publication is important work on checking and verifying Lean, but does not establish a complete, version-matched, verified replacement normalizer for this project [17].

This changes the engineering order. Use the native proof-producing machinery for suitable closed observations and realized recursive definitions. Keep the existing kernel reduction path where it works well. Add residual scheduling and observation decomposition around these backends, rather than starting by reproducing every kernel reduction feature.

A proof-producing evaluator may prove  $e = v$  without  $e$  and  $v$  being definitionally equal under the desired reduction strategy. That does not make it an under-approximation of conversion. It makes it a semantics-preserving rewriter with explicit evidence. If conversion itself is the intended specification, choose a smaller fragment whose reductions are genuinely kernel-convertible, and have the checker validate that property. Do not switch between these claims in a performance argument.

For an open branch, a backend that reports false must support (1). A useful implementation exposes three responses:

```
proved(value, equalityProof, dependencies)
refuted(branchCertificate, dependencies)
unknown(residual, blockers, reason)
```

These are conceptual interfaces, not asserted Lean API declarations. A branch certificate abstracts the holes into a well-typed telescope; it is not an unclosed proof term containing unassigned metavariables.

Observation decomposition must be proved too. Splitting a conjunction is straightforward. Translating a Boolean `BEq` test into propositional identity is not justified without lawfulness. For a user-defined test, retain its exact Boolean semantics unless a checked equivalence theorem licenses the transformation.

**Implementation consequence.** Initially let custom evaluation guide goal ordering and candidate priority, while only existing checked paths prune. Then add certificate production for a small fragment of constructors, applications, projections, and specific recursor equations. Differential testing on generated programs is a valuable bug detector, but even  $10^5$  successful tests is not a proof of conservativity. Include tests with open holes, delayed assignments, rollback, universe constraints, and proof-dependent types; closed-term agreement alone misses the hardest failure modes.

#### 4.4 R3.4: does top-down angelic recursion admit correctness guarantees?

**Answer.** Yes under explicit boundedness and completeness assumptions. Top-down organization is not itself an obstacle. The difficult parts are preserving all possible recursive results, enforcing their eventual consistency, and not confusing an incomplete oracle solver’s failure with impossibility.

A recursive call on an unknown smaller input can be interpreted by a symbolic result

variable. Calls with the same complete argument tuple should share the same variable when their deterministic semantics requires it. The tuple must include accumulators and relevant environment values. Omitting that correlation may be a safe but very weak over-approximation; imposing a guessed correlation between unequal calls can incorrectly prune.

Known observations can constrain the oracle when the contract actually entails them. If  $C(f)$  implies  $f(v) = w$ , the oracle may use  $w$  for that call while searching for a function satisfying  $C$ . A contradiction obtained this way refutes *contract-satisfying* completions. It need not show that every arbitrary completion evaluates the observation to false. The certificate should therefore retain the contract implication explicitly.

**Research evidence.** Burst supplies angelic synthesis with strengthening and backtracking. Its completeness statements depend on the behavior of the angelic synthesis subroutine and the validity of failure information; they are not an unconditional termination theorem for arbitrary recursive Lean synthesis [18].

**Proposition 4.2** (A sufficient bounded top-down setting). *Fix a finite candidate-body grammar, a finite admissible oracle-call/value space, a well-founded recursive-call relation, an exact terminating solver for the resulting constraints, and a fair traversal that resolves each finite case once and stops on exhaustion. If refinement excludes only infeasible interpretations or invalid completed candidates, top-down angelic search terminates and is complete for that bounded candidate problem.*

*Proof.* There are finitely many candidate bodies and finitely many oracle assignments for each. Exact constraint solving and well-founded evaluation resolve each case. Valid refinement removes no successful body with its genuine recursive results. Fair traversal therefore either reaches such a body or exhausts all possibilities.  $\square$

Finite syntax alone does not guarantee the other assumptions. Recursive calls may involve large values or new arguments, and solver timeouts remain unknown. For an initial implementation, use angelic assumptions only to prioritize search, then actually evaluate and certify each completed body. Promote angelic infeasibility to hard pruning only when the relevant finite relaxation has a checked or otherwise established complete refuter.

## 5 R4: dependent motives, generalization, and transport

### 5.1 R4.1: can a useful motive class be enumerated completely?

**Answer.** Yes for a precisely specified finite syntactic class. No, abstraction of maximal index occurrences alone is not closed under the inventions required for accumulators and strengthened results.

Suppose a goal contains finitely many selected maximal occurrences of index terms. Every subset determines a candidate abstraction, after scope and typing checks. Enumerating all subsets terminates and is complete for *that construction*. Most candidates may fail to typecheck; that is an efficiency issue, not a reason to infer completeness for a broader class.

An accumulator parameter  $S$  may not occur in the goal's indices at all. Similarly, a strengthened result  $T \times U$  may introduce  $U$  for the first time. Neither can be produced solely by replacing existing index occurrences. Extend the class with bounded dependent carrier templates and *dependency-closed generalization sets*. If a local is reverted, all later locals whose types or values depend on it must be handled in telescope order.

For vector map, a natural motive is

$$M(n, x) = \text{Vec } B \, n.$$

The function argument  $A \rightarrow B$  may remain a fixed parameter. In an accumulating variant, a useful motive might instead be

$$M(n, x) = \Pi s : S, T(n, x, s).$$

The second is not merely another subset of the first goal's index occurrences. A unified motive grammar makes the relationship to carrier invention explicit.

Compound and repeated indices need special treatment. A major premise such as  $\text{Vec } A(n + m)$  cannot always be handled by pretending that  $n + m$  is an independent local variable and forgetting its defining equation. Generalize the compound expression to a fresh index, retain the equality and its dependent consequences, and let native elimination construct the resulting obligations.

**Implementation consequence.** Enumerate motives in stages: native inferred motive; native motive after minimal dependency-closed reversion; bounded extra parameter or product strengthening; finally a wider syntactic abstraction grammar. Record the stage and bound. Use a fair fallback to preserve relative completeness; otherwise explicitly report a heuristic search rather than “no motive exists.”

## 5.2 R4.2: how can transports be handled without destroying identity?

**Answer.** Treat transports as typed edges with checked equality evidence. Normalize only through justified rules, and distinguish proof-irrelevant evidence from the type-changing operation that evidence enables.

The proposal's example  $\text{Vec } A(n + 0)$  versus  $\text{Vec } A n$  does not require a propositional cast: Lean reduces addition on its second argument.  $\text{Vec } A(0 + n)$  is a suitable replacement when  $n$  is a variable; proving  $0 + n = n$  is then relevant [16].

For a family  $F : I \rightarrow \text{Sort } u$  and  $e : i = j$ , transport maps  $F(i)$  to  $F(j)$ . Proof irrelevance can identify different proofs of the same proposition; it does not identify the two endpoint types, erase the family, or justify dropping the transport from a term that must typecheck. A printed presentation may hide a cast, but the replayable artifact must preserve it.

A practical representation keeps an ordinary Lean expression plus an auxiliary transport record containing source type, target type, equality proof, and normalization provenance. For arithmetic indices, orient checked equations into a chosen normal form and reuse a cached equality certificate. Eliminate identity casts and compose compatible paths when a proved rule permits it. Do not enumerate arbitrary equality proofs: synthesize the equation once, then let proof irrelevance and typed proof caching remove redundant evidence.

There is no general cheap canonical quotient of all dependent Lean programs by propositional equality. Setoids change the specification and require their own compatibility proofs; heterogeneous equality changes the form of the obligations but does not automatically construct the desired homogeneous inhabitant. Both can be useful in specialized intermediate theories, not as a universal shortcut around native dependent typing.

**Implementation consequence.** Bound nontrivial transport steps and assign them a nonzero structural penalty unless a certified normalization removes them. The proposal to make all casts zero-cost creates avoidable cycles and makes a size objective misleading. For display, provide a readable expression and an exact checked expression; do not use the readable form as the identity key.

## 5.3 R4.3: what unfolding policy should search use?

**Answer.** Use staged, demand-driven unfolding with an explicit policy in the state and cache keys. Neither global normalization nor a single maximal transparency setting is a convincing

default for large dependent search.

**Research evidence.** Canonical’s 2025 account describes unfolding certain aliases that expose function types, reducible definitions, and typeclass-related structure as part of translation into its own representation [5]. That is concrete evidence for selective unfolding, not proof that the same translation policy is optimal inside Lean’s metaprogramming engine. The sauto publication establishes a relevant dependent type-inhabitation search approach [19]; the precise current implementation-level unfolding policy was not verified in this investigation, so it should not be represented as a confirmed benchmark result or copied by attribution alone.

The proposed native policy has three stages. First perform cheap weak-head normalization sufficient to expose a function, constructor, or indexed-family head. Second, when matching is blocked on a reducible named definition, unfold only the demanded head under a fixed budget. Third, permit a bounded stronger normalization or theorem-based rewrite for a specific failed match. An opaque constant should remain opaque unless the permitted interface supplies a checked equation or another explicit reason to expose it.

Record why unfolding was requested: exposing a binder, determining a recursor, matching an index, or evaluating an observation. These requests have different cost/benefit profiles. Cache a result only under the same environment, transparency options, local context, and assignment dependencies. Do not let a failed low-transparency match become a permanent negative fact about the term.

**Implementation consequence.** Compare three policies on the same dependent cases: fixed weak-head normalization, staged demand-driven unfolding, and bounded up-front normalization. Measure successful matches, reduction work, and missed candidates at fixed logical budgets. Treat broader comparative claims about Canonical or sauto as experiments to perform, not as existing evidence of superiority.

## 5.4 R4.4: is native motive computation available as a library?

**Answer.** Yes. The inspected Lean 4.34 source exposes useful components, but they have concrete preconditions and are not a universal motive-enumeration API.

The public native operation includes

```
Lean.MVarId.induction
  (mvarId : MVarId) (majorFVarId : FVarId)
  (recursorName : Name) (givenNames : Array AltVarNames := #[]) :
  MetaM (Array InductionSubgoal)
```

The result includes each subgoal, its fields, and a free-variable substitution. The same file exposes `getMajorTypeIndices` and `mkRecursorAppPrefix`. The latter builds the recursor prefix and computes a motive by abstracting the major premise and indices. The index-extraction path checks that indices are suitable distinct variables and reports specific failures when they are not. Native induction reverts dependencies and reintroduces the major premise and indices [20].

Wrap these operations in a version-pinned adapter. Each attempt runs in a fresh saved state, creates no globally visible declaration, and returns a structured outcome: success with subgoal substitutions; compound-index precondition failure; target-sort/elimination restriction; failed unification; or resource exhaustion. Preserve interrupts and do not classify a timeout as an impossible motive.

For a compound index, first use an appropriate native generalization path rather than invoking the low-level operation with violated preconditions. After native induction, retain its substitutions instead of assuming that original free-variable identifiers still denote the branch

locals. This is especially important when attaching residual examples to new branch holes.

**Implementation consequence.** The first dependent milestone should be vector map and a small family of indexed constructors using this adapter, not a replacement recursor compiler. Benchmark the cost of both successful and failed attempts. “Cheap failure” is a performance target to measure; it is not guaranteed merely because the function is callable from MetaM.

## 6 R5: recursive bodies and checked realization

### 6.1 R5.1: does a capability interpreter agree with the realized definition?

**Answer.** Agreement must be proved for the chosen body language and realization. For well-founded recursion, expect a propositional equation rather than assuming a convenient definitional reduction. Native equation-based evaluation is the first implementation to try.

A dependent capability has the form

$$rec_x : (y : \alpha) \rightarrow R(y, x) \rightarrow C(y),$$

where  $R$  is well-founded and  $x$  contains every argument relevant to recursive state. A body functional has type

$$F : (x : \alpha) \rightarrow ((y : \alpha) \rightarrow R(y, x) \rightarrow C(y)) \rightarrow C(x).$$

An accumulator is not an invisible interpreter variable: either it belongs in  $x$ , or the result family explicitly quantifies over it. This prevents oracle memoization and termination checking from silently treating unequal calls as the same call.

Compile a completed, typed body using a chosen native realization, obtaining  $f$  and its checked equation

$$f(x) = F(x, \lambda y h. f(y)). \quad (4)$$

The exact equation may include the elaborator’s administrative arguments and casts. The native recursion documentation explicitly distinguishes convenient structural reduction from the potentially non-definitional equations of well-founded recursion [16].

A direct interpreter can be related to  $f$  by well-founded induction. At each recursive call, apply the induction hypothesis to the smaller argument and use congruence for the body constructs. This proves agreement if the body interpreter preserves the semantics of every construct and the realization has equation (4). Neither condition follows merely from using similar-looking source code.

A fuelled interpreter needs a stronger statement:

$$\text{run}(n, x) = \text{some}(v) \implies f(x) = v.$$

Its failure to produce a value with fuel  $n$  does not imply that  $f(x)$  has a wrong value. The fuel bound may be inadequate even for a terminating function. Return unknown, retain the residual, and charge the work.

**Implementation consequence.** Reuse the proof-producing evaluation path of R3.3 on realized candidates and their equations. For open-body exploration, interpret only a small typed capability language, initially without hard pruning from uncertified results. Realize promising completed bodies once, cache their checked equations, and certify the contract. Never insert a global temporary axiom naming the recursive function and later forget to remove it from the accepted dependency closure.

There is also a distinction between mathematical acceptance and deployment: a candidate can kernel-check yet encounter compilation or binding problems in the front end. Since the proposal records such an example for a synthesized recursor term, publication should include a separate executable-binding check when the requested output is an executable definition [3].

## 6.2 R5.2: what changes when angelic execution is top-down?

**Answer.** The semantics of the relaxation can remain the same; the representation of alternatives and the way constraints propagate change. Top-down search trades the compact bottom-up representation of many terms for goal-directed refinement of a typed sketch.

A top-down node contains a body skeleton, its holes, recursive-call variables, and constraints derived from observations. Filling a constructor hole can immediately propagate a required result to its fields. Filling a guard can select which branch constraints apply. Filling a recursive call's arguments can identify a previously symbolic call with an observed input or another call. These changes naturally fit the dependency watchers of R1 and R3.

The challenge is retaining correlated alternatives. If one observation demands that a recursive result be either zero or one, and another demands that the same call be positive, prematurely choosing zero loses the feasible option. Store the alternatives symbolically, or branch over all of them within the finite bound. A solver that only tries one satisfying angelic assignment can still be a useful proposer, but its later failure does not refute the body.

Burst's bottom-up method uses finite-tree-automaton representations and a particular refinement organization [18]. A top-down implementation cannot inherit the space savings or the proof assumptions simply by using the word "angelic." It needs its own constraint representation and the bounded correctness conditions stated in R3.4.

**Implementation consequence.** Start with recursive calls at already observed inputs, then symbolic unknown results used for ranking, and finally exact bounded oracle constraints that may support pruning. Keep actual closed execution and kernel-certified contracts as the acceptance path. This gives useful intermediate milestones without making the entire search rewrite depend on a general angelic solver.

## 6.3 R5.3: is there a small canonical recursion basis with measured coverage?

**Answer.** There is a small useful engineering basis, but no coverage percentage for Leant2 follows from that fact. A universal semantic basis and an efficient search basis are different objectives.

A proposed staged basis is: structural recursion on one major premise; structural recursion with generalized parameters or tupled results; matching additional arguments inside that recursion; bounded course-of-values access; and well-founded recursion under a small measure grammar. Mutual or nested families can initially use native elaboration when an appropriate body sketch has been found, rather than expanding every generated recursor in the inner loop.

Several textbook examples need less machinery than their surface form suggests. Zip can recurse on one list while matching the other. Fibonacci can use the pair state in R2 rather than directly invoking two sub-subterm calls. Tree flattening can fold into difference lists. Merge can use an explicit measure such as the sum of the two list lengths, whereas a search that insists on a fixed single structural argument may have difficulty. These examples motivate the tiers; they do not establish empirical coverage on a corpus.

A body capability should encode exactly which calls are available. A "course-of-values depth two" restriction is a syntactic capability bound, not a generic theorem about all course-of-values definitions. A measure grammar must also bound arithmetic expressions, lexicographic tuples,

and proof search for decrease. Otherwise a finite list of scheme names hides an unbounded secondary synthesis problem.

**Implementation consequence.** Classify reference programs before measuring synthesis: minimum sufficient scheme tier, whether a carrier is supplied, whether helpers are supplied, whether examples are trace-complete, and whether a universal proof is required. Run both provider-enabled and provider-withheld tracks. Report representation coverage separately from search success; a program may be expressible in tier one yet not found within the budget. No numerical coverage claim is made here.

## 7 R6: contracts, induction, and proof debt

### 7.1 R6.1: how should value holes and proof holes be interleaved?

**Answer.** Use dependency-triggered proof tasks with a cheap eager tier and a bounded expensive tier. Waiting for the entire program is too late, but declaring every proof goal meaningless while it contains holes is too strong.

A proof obligation can simplify or produce useful equalities before all its value dependencies are known. A proposition such as  $x = x$  needs no value of  $x$ ; constructor discrimination can refute some partial branches immediately. Other obligations genuinely block on a value choice. Represent that distinction with dependencies rather than a blanket “proof last” rule.

For each proof task, retain its exact context, target, relevant value holes, attempted tactic tier, and certified consequences. Cheap normalization, reflexivity, and explicit contradiction checks run whenever a dependency changes. More expensive tactics run only after the relevant branch values are stable enough or the candidate has survived inexpensive tests. A timeout suspends the task; it does not certify that the chosen value is unprovable.

**Research evidence.** Synquid demonstrates synthesis directed by polymorphic refinements with automated constraint solving [21]. Its logical fragment and solver assumptions are not those of arbitrary Lean propositions. The analogue proposed here is an explicit finite abstraction of selected predicates, with all consequences justified by Lean proofs rather than silently assuming a complete SMT interpretation of Prop.

A useful optional layer maintains abstract predicates such as length bounds, non-emptiness, or membership facts. Each transfer rule has a checked lemma. The abstract domain can be incomplete: failure to infer a property merely leaves an obligation. This supplies a Lean-native refinement discipline without requiring a complete decision procedure for every contract.

For subtype targets, synthesize a branch value and its local proof together. A recursive result then carries the induction invariant needed by the next branch. Reuse that proof rather than reconstructing a global theorem about an opaque recursor expression after synthesis has finished.

**Implementation consequence.** Make proof debt a first-class queue over the same dependency graph as value search. Wake a task only when its blockers change or its allocated tier increases. Record whether a proof was attempted before or after the relevant value became stable; this exposes wasted repeated work and delayed failures in a way that a single aggregate proof timer cannot.

### 7.2 R6.2: which properties have a recognizable induction-along-the-program proof?

**Answer.** A useful sufficient class can be recognized through generated verification conditions and a certified leaf solver. There is no need to decide membership in the much broader class of

all properties for which a clever induction, generalization, or auxiliary lemma exists.

For a fold  $s(xs) = \text{foldr } \delta \ z \ xs$ , let  $I(xs, s)$  be a supplied or synthesized invariant. Generate the obligations

$$I([], z), \tag{5}$$

$$\forall a, xs, r, I(xs, r) \rightarrow I(a :: xs, \delta(a, r)), \tag{6}$$

$$\forall xs, r, I(xs, r) \rightarrow C(xs, \phi(r)). \tag{7}$$

The input  $xs$  remains in the invariant, so this scheme expresses relationships between the input and the accumulated state, not only properties of the state.

**Theorem 7.1** (Local invariant certification). *Proofs of (5)–(7) imply  $\forall xs, C(xs, \phi(\text{foldr } \delta \ z \ xs))$ .*

*Proof.* Induction on  $xs$  proves  $I(xs, \text{foldr } \delta \ z \ xs)$ . The empty case is (5); the cons case follows from the induction hypothesis and (6). Apply (7) to obtain the contract.  $\square$

This theorem provides a concrete proof template. The same construction can be internalized by a motive

$$xs \mapsto \{r : K \mid I(xs, r)\},$$

so that each recursive branch returns a value and the required evidence. Projecting the value yields the program while the proof projection establishes its invariant.

For map with element function  $f$ , choose  $I(xs, ys) \equiv |ys| = |xs|$ . The cons verification condition reduces to equality of successor lengths after using the induction hypothesis. For a numerical accumulator, an affine invariant may reduce the step to linear arithmetic. For reverse or sorting, an adequate invariant may require append, permutation, or ordering lemmas; those are additional inputs to the proof template, not consequences of the word “structural.”

Define a recognizer for a sufficient fragment by allowing finitely many invariant templates, a terminating oriented rewrite system with proved rules, and a checked decision procedure for the residual arithmetic or constructor formulae. Acceptance by this recognizer produces a proof. Failure means only that the selected template/solver combination did not certify the property. Even when every leaf lies in a decidable theory, discovering the right invariant can remain the difficult synthesis task.

**Implementation consequence.** Expose a separate outcome *template applicable but invariant missing*. This can trigger R2’s generalization or helper proposals. Do not repeatedly spend the largest proof budget on a monolithic goal whose branch obligations show that the current invariant cannot connect the recursive states.

### 7.3 R6.3: what Lean induction automation is usable now?

**Answer.** Use native induction and functional-induction infrastructure to choose a small set of explicit proof skeletons, then apply a bounded leaf portfolio. Do not assume that a general-purpose tactic automatically discovers all useful induction variables, generalizations, and lemmas.

The inspected native induction interface already supplies recursor-based subgoals and substitutions, as explained in R4.4. Lean’s reference manual also documents functional induction associated with recursive definitions [16]. These are appropriate infrastructure for a proof that follows the synthesized program’s recursion. The choice of an adequate invariant remains the synthesizer’s responsibility.

A practical core-first sequence is: introduce quantified inputs; select the major premise or the realized function’s induction principle; apply the known invariant/generalization template;

simplify with the program’s checked equations and local induction hypotheses; then try the relevant arithmetic or congruence solver. Guard every call by explicit work limits and preserve the state on failure. `grind` belongs in the leaf portfolio, not in the design as an unspecified promise of automatic lemma discovery.

**Research evidence.** Canonical is directly relevant to automated dependent inhabitation and recursor-based reasoning [5]. LeanHammer’s current work supplies learned premise selection and proof reconstruction infrastructure, including a 2026 conference publication [22]. These are distinct resources: neither a theorem-retrieval accuracy figure nor success on ordinary proof corpora establishes automatic induction coverage for newly synthesized programs.

External hammers and cloud retrieval should remain optional extensions, not a hidden dependency of the core-only path. Their availability, latency, and axiom effects must be included in the receipt. A synthesized proof term that reconstructs inside the permitted profile is valuable; a timeout or failed external reconstruction is simply unresolved.

**Implementation consequence.** Build a regression suite with hand-classified proofs: one induction plus simplification; induction plus arithmetic; generalization required; helper lemma required; stronger invariant required. This is more informative than one overall solved count and permits targeted expansion of the automation without changing the semantics of an accepted result.

## 7.4 R6.4: is there a principled policy for proof debt?

**Answer.** There is a principled decision-theoretic objective, but no reason to claim that ordinary independent-arm bandit theory yields an optimal policy for this coupled search. Proof tasks share candidates, lemmas, contexts, and setup work.

A reasonable heuristic score is

$$\text{priority}(j) = \frac{\widehat{\Pr}(\text{useful certification or refutation} \mid j) \cdot \widehat{\Delta U}_j}{\widehat{\text{work}}_j + \widehat{\text{switch}}_j + \varepsilon}.$$

The utility can reward a better certified candidate, the rejection of a large branch, or a reusable local lemma. Estimates should be learned or calibrated from held-out traces. This formula is a proposed heuristic, not a theorem of optimal scheduling.

Use deterministic epochs and minimum service quotas. Each epoch assigns work to value expansion, residual evaluation, and proof tasks. Within the proof queue, prefer stable local obligations over large global ones, and reuse a warm context when doing so saves real setup work. Escalate a task from cheap to expensive tactics only when its previous tier has completed and a relevant reason exists to spend more. A fair quota prevents uncertain but essential proof obligations from starving forever.

The key scheduling unit is a *task continuation*, not necessarily an entire tactic invocation that restarts from scratch. Some native tactics may not expose resumable internals; account for that honestly and cache their completed preprocessing where possible. Do not implement “500 milliseconds per proof” and then claim deterministic results. Prefer logical limits tied to the pinned evaluator/tactic version, with wall-clock cancellation as a separate status.

**Implementation consequence.** Log the number of obligations created, woken, discharged, refuted, retried without changed dependencies, and timed out. Compare a fixed staged policy with learned prioritization under equal total work. Include total candidate quality, not just tactic success rates: cheaply proving properties of low-value candidates can otherwise look artificially good.

## 8 R7: library retrieval with dependency and type classes

### 8.1 R7.1: what abstraction is sound and useful for dependent signatures?

**Answer.** Use a progressively refined typed hypergraph, retaining constructor and index shape and treating unknown information as an over-approximation. A graph of isolated type heads is a useful first ranking feature, but not a complete synthesis semantics.

A component  $c : A_1 \rightarrow \dots \rightarrow A_k \rightarrow B$  needs all of its arguments, not merely one edge from an available input head to  $B$ . Use a hyperedge whose premises record the argument abstractions and relevant shared type variables. Ordinary Lean contexts permit reusing variables, so the abstraction must also model contraction rather than accidentally treating every value as a linear token. Lambda introduction, locally available functions, constructors, and permitted elimination rules need their own abstract transitions if the abstraction is to justify negative reachability.

An initial abstract type can retain: the outer constructor; the arrangement of type arguments to bounded depth; an equality partition of repeated type variables; coarse index forms such as zero/successor/unknown; and necessary universe relationships. Erase an unresolved index to an explicit top element, not to a guessed constant. Preserve distinctions that explain common failed matches:  $\text{Vec } A \ n$  and  $\text{Vec } A \ (n+1)$ , or a function whose input and output share the same type parameter versus one with unrelated parameters.

When a proposed composition fails concrete Lean checking, refine only the abstraction responsible for the spurious connection. A term-index mismatch can split an index class; a missing dictionary can add a class precondition; a universe contradiction can distinguish level patterns. A timeout supplies no such counterexample and must not be used as a refinement fact.

**Proposition 8.1** (Safe abstract non-reachability). *Suppose every concrete construction rule in a selected grammar is simulated by an abstract transition, and every concrete initial resource maps to an abstract initial resource. If the abstract target is unreachable, no concrete term in that grammar reaches the target.*

*Proof.* Induct on a concrete term’s construction derivation. Initial resources are represented by assumption. Every subsequent construction maps to an abstract transition whose premises are reachable by the induction hypothesis. Thus any concrete solution would induce an abstract path to the target, contradicting non-reachability.  $\square$

The simulation assumption is the whole issue. A depth-three library path filter that omits recursors, lambdas, casts, or newly invented helpers may still rank providers effectively, but it cannot establish nonexistence for a search that uses those constructs. Likewise, erasing universes is safe only when it enlarges the possible matches rather than introducing a restrictive default.

**Implementation consequence.** Initially use abstract reachability to prioritize a local provider set, with a wider deterministic fallback. Promote negative results to hard pruning only for a documented finite component sublanguage with a simulation argument. Measure selectivity and concrete mismatch causes; a universal type-variable node that connects everything signals a need for refinement, not a reason to silently discard inconvenient polymorphic functions.

### 8.2 R7.2: has type-guided refinement already handled type classes?

**Answer.** Yes. Hoogle+ explicitly represents typeclass constraints by dictionaries. The remaining research problem is extending the approach to Lean’s dependent and universe-sensitive setting, not inventing dictionary treatment from scratch.

**Research evidence.** Section 5.1 of Hoogle+’s paper describes dictionary types, extra dictionary arguments for constrained methods, and instance functions that construct result dictionaries

from prerequisite dictionaries. It also states limits on higher-kinded type variables [23]. This directly answers the proposal’s question about whether type-guided abstraction refinement has been tried with type classes, while leaving important differences from Lean unresolved.

For example, abstract a constrained operation schematically as

$$eqMethod : EqD(A) \rightarrow A \rightarrow A \rightarrow \mathbb{B}.$$

An instance for lists is another component

$$eqList : EqD(A) \rightarrow EqD(List\ A).$$

The reachability problem now includes obtaining dictionaries. In Lean the actual dictionary terms, local instances, parameters, and output-parameter behavior must be interpreted by native instance synthesis rather than by an independent approximation that silently becomes authoritative.

The proposed cache key “class and type” is insufficient in general. Add the local instance context, environment fingerprint, universe instantiation, relevant options, and dependencies on unresolved metavariables. An instance search that is stuck on an output parameter is not a cached global failure. A successful cached dictionary should be reinstantiated and typechecked in the new context; proof-bearing or data-bearing class fields remain part of the actual term.

**Implementation consequence.** Index constrained components with explicit dictionary prerequisites, then resolve concrete obligations lazily once their dependent parameters are sufficiently known. Cache positive dictionaries aggressively under validated keys. Keep negative instance information local to the complete, fixed search state and bound that justified it. Benchmark instance-search work separately from ordinary provider matching, since an apparently small provider set can still open a large instance search.

### 8.3 R7.3: can premise selection consume data goals and examples?

**Answer.** The current Lean interface can be called on a data goal and has a hint field, but this does not imply that an installed selector understands examples or has been trained for program-component relevance. A dedicated adapter and training/evaluation corpus are needed.

The inspected Lean 4.34 API has

```
Selector := MVarId -> Config -> MetaM (Array Suggestion)
```

with configuration fields for a suggestion limit, caller, filter, and an optional arbitrary hint. Its type does not restrict the goal to Prop; the hint is explicitly not guaranteed to be used by a selector [24]. Consequently an example summary can be passed through an adapter, but claiming that this automatically supplies behavior-aware retrieval would be unwarranted.

**Research evidence.** The current cloud premise-selector documentation describes goal-state and premise embeddings, and warns about substantial cold-start costs. It is therefore not a dependable unbudgeted operation inside a short core-only query [25]. Its documented interface and the LeanHammer research motivate a reusable retrieval backend, not an established score for carrier or data-program synthesis.

Construct a program-retrieval corpus from executable definitions in Std, Mathlib, and appropriately licensed libraries. Hide the target body; collect its signature, permitted environment, generated or specified observations, and the dependency components actually used by its implementation. Multiple implementations may be valid, so dependency prediction is a training signal, not a unique ground truth for intended behavior.

Leakage control is essential. Remove the target declaration itself, aliases, and providers that transitively expose its body when evaluating genuine synthesis. Split related function families and dependency strongly connected components between training and test where possible. Otherwise a held-out “definition” may be recoverable by an imported synonym. Separate provider-enabled API search from provider-withheld algorithm invention.

For behavior features, compute summaries such as length change, shape change, order preservation on observed inputs, and use of supplied callbacks. These are relevance signals, not logical filters unless certified. A component that fails to equal the final answer may still be an essential intermediate step.

**Implementation consequence.** Keep the existing selector interface for proposition goals and implement a data-goal wrapper that combines type compatibility, abstract reachability, and behavioral features. Cache indexes per environment. The default lane should remain local and deterministic; optional remote retrieval must use a recorded result manifest and a separately charged latency budget.

## 9 R8: ranking, equivalence, and ambiguity

### 9.1 R8.1: what formal objective should rank inhabitants?

**Answer.** A frozen minimum-description-length objective is defensible as an explicit prior, not as a universal definition of user intent. Correctness is a hard constraint; program quality is a separate, testable preference model.

Choose a prefix-decodable or otherwise explicitly weighted grammar and define

$$J_Q(p) = L_{\mathcal{G}_Q}(p) + \lambda_1 \text{casts}(p) + \lambda_2 \text{helpers}(p) + \lambda_3 \text{risk}(p).$$

Here  $L$  counts the complete representation, including carrier, finisher, and helper definitions, not just the visible outer term. A library call can be cheap if the grammar assigns it a short code, but this is a stated prior over the available library, not an objective fact that the called implementation is computationally simple. Any runtime or space cost model should be an additional explicit objective with its own evidence.

For finite examples  $E$ , an MDL-style interpretation is

$$p^* \in \arg \min_{p: p \models E} L_{\mathcal{G}_Q}(p).$$

For a universal contract, replace the sample constraint with the existence of a checked proof. Neither formulation says that the user intended the smallest consistent program. It does make the ranking criterion reproducible and inspectable.

**Research evidence.** Learned ranking has empirical precedent in practical programming by example, including the real-time ranking work of Natarajan and colleagues [26]. User interaction for disambiguating consistent programs has also been studied [27]. These are reasons to evaluate ranking against user intent and ambiguity; they do not establish that MDL or Leant2’s unused-input heuristic is optimal for dependent proof-assistant programs.

Unused-input penalties are especially delicate. A correct constant function or projection may intentionally ignore arguments. Syntactic occurrence is not information flow:  $x - x$  mentions  $x$  but, under ordinary arithmetic semantics, is constant. Conversely, a dependency hidden in a callback or dictionary may be operationally important. Use such penalties as weak, documented priors, not a first-key rule that always dominates every other quality measure.

To claim the best candidate is optimal under  $J_Q$ , every unexplored search state needs a valid lower bound on the same frozen objective, and the search must not have discarded alternatives

through incomplete pruning. The stopping condition is then that the frontier lower bound is no smaller than the best candidate cost, with a tie policy appropriate to the desired result set. A grace-period cutoff yields only *best seen*, not this certificate.

**Implementation consequence.** Publish a total deterministic order: certification class, frozen cost, and a canonical typed-expression tie-breaker. Evaluate held-out behavioral accuracy, complexity, readability, and user preference separately. Reference solution rank is useful, but allow proved equivalent implementations and more than one reasonable intended solution.

## 9.2 R8.2: which candidate equivalence is cheap and meaningful?

**Answer.** Use several explicitly labeled relations. No single cheap test provides both general extensional equality and dependable canonicalization for all dependent Lean terms.

The first layer is typed alpha-canonical structural identity, including the relevant universes, binders, and context abstraction. The second is native checked definitional equality. A third layer uses explicit equality proofs, possibly pointwise proofs and function extensionality when permitted by the profile. A fourth layer groups candidates that agree on a finite probe set. Only the last layer should be called an *observational cluster*, not a proved equivalence class of programs.

The current engine’s printed-form deduplication is useful for avoiding visual duplicates, but the printer is not an identity function: it can hide implicit arguments, universes, and coercions [2]. Replace its use in semantic bookkeeping with an exact typed key. Keep readable-print equality only as a display convenience, and make hidden distinctions available in the artifact.

**Example 9.1** (Observational coincidence is not a general congruence). Let  $f(n) = n$  and let  $g(n) = n$  for  $n < 2$ , with  $g(n) = 0$  otherwise. They agree on probes  $\{0, 1\}$ . A surrounding program that applies its function argument to 2 distinguishes them. Therefore a synthesizer cannot replace  $g$  by  $f$  or vice versa in arbitrary higher-order contexts merely because their current probe signatures match.

The same issue arises in recursion when a surrounding program creates new recursive inputs, or after CEGIS introduces a new counterexample. For a restricted first-order straight-line DSL, observational quotienting can be valid when every allowed operation acts pointwise on the retained observation vectors and the relation is proved to be a congruence. The optimization is valuable in that setting; its preconditions must not be omitted when moving it into Lean’s higher-order language.

Dependent functions introduce a further issue: observations at different arguments may inhabit different fibers. Preserve the argument and type with every observation, and compare outputs only by a well-typed relation. Proof irrelevance can simplify proof components of the same proposition; it does not remove the need to align data indices and transports.

**Implementation consequence.** Keep one displayed representative per observational cluster while retaining alternative implementations internally unless safe quotienting is established. Search for a small distinguishing input between different clusters and certify its output difference. Failure to find one within a bound means only that the bounded test did not distinguish them, not that the functions are equal.

## 10 R9: learned proposals without expanding the trust boundary

### 10.1 R9.1: what accuracy can be claimed for model-proposed carriers?

**Answer.** No numerical accuracy for the proposal’s exact task is established by the sources investigated here. Proof-retrieval results and general code-generation benchmarks do not

measure carrier or helper invention from only a Lean type and finite examples. The appropriate answer is a controlled experiment with precisely defined success criteria, not an extrapolated percentage.

Use the E8-style suite and the thirteen stretch cases as a starting point, but avoid measuring only recovery of familiar textbook names. Include renamed symbols, held-out function families, different encodings, and unfamiliar combinations of known operations. Separate prompts containing only types and examples from prompts containing a full symbolic specification or a reference algorithm. The latter provide substantially different information.

For each model and fixed proposal budget, record four outcomes: well-formed and well-typed proposal; a compatible implementation found under that proposal; a kernel-certified solution to the original query; and improved end-to-end quality or latency after charging model generation and validation. A proposed carrier can be mathematically suitable yet difficult for the downstream search, so carrier validity and end-to-end success should not be conflated.

Nor is exact agreement with a named reference carrier the right correctness criterion. R2 shows that several representations can implement the same function. Evaluate the complete carrier-seed-step-finisher solution under the frozen objective, and report whether a carrier was merely plausible or actually witnessed by a certified implementation.

Compare at least a deterministic grammar baseline, failure-driven symbolic proposals, and the same search with model proposals added at fixed checkpoints. Use identical downstream budgets and provider availability. Report per-family outcomes and uncertainty rather than a single small-sample percentage. This report does not run that model experiment, and therefore makes no claim about current carrier-proposal accuracy.

## 10.2 R9.2: what makes learned guidance deterministic and auditable?

**Answer.** Deterministic validation and replay of a frozen proposal artifact are more important than hoping that model generation itself is bitwise deterministic. A seed and temperature zero are not a sufficient reproducibility contract.

Record a content-addressed manifest containing the exact prompt input, model and tokenizer identifiers, available runtime details, decoding parameters, returned proposals, ordered alternatives, validation results, and the frozen Lean environment and grammar fingerprints. Replay uses the recorded proposal bytes and their parsed typed representations; it does not call the model again. Canonical ordering and validation occur locally under the pinned implementation.

The proposal language should be restricted data: carrier templates, references to permitted constants, bounded helper signatures, or typed expression skeletons. Do not treat arbitrary model-produced Lean commands or metaprograms as safe merely because a final theorem will be checked. Elaboration-time code can perform effects outside the mathematical kernel. A whitelist or a narrow parser keeps proposal evaluation inside the intended boundary.

A model that only reorders an already fixed finite grammar preserves its candidate universe, but it can still change budget-limited results. In an infinite enumeration it can also cause starvation unless fairness is enforced. If a model introduces a helper or carrier outside the recorded bounded grammar, then the grammar has changed. Report that extension explicitly rather than claiming unchanged bounded completeness. Alternatively, require every accepted proposal to compile back into the original bounded grammar.

The audit record should connect each published term to its exact query, contract proof, helper closure, axiom inventory, proposal manifest, and search checkpoint. The mathematical proof remains valid independently of the model; the receipt explains how the program was found and what search or ranking claims are justified.

**Implementation consequence.** Keep the default engine offline, core-only, and deterministic. An optional proposal source should enter the same validated interface as enumerated proposals and receive a bounded share of work. Do not add special acceptance privileges for model-generated terms, and do not require the model to replay an already certified result.

## 11 An integrated Lean-native architecture

### 11.1 Keep authority at the original query

The preceding answers do not require a second trusted type theory. Native Lean expressions, local contexts, and metavariable states remain authoritative. A separate residual or abstract representation is permitted for speed, but its negative conclusions need a justified connection back to the native query.

The request-binding layer should retain the frozen target and contract rather than accepting a caller-supplied pair of arbitrary types as sufficient context. The inspected engine already constructs contract proofs by extracting the property of a synthesized subtype; the gate then checks the supplied program and proof types [2]. Preserve that connection explicitly in the proposed interfaces, especially as more proposal sources and realization paths are introduced.

There is a specific subtlety in the existing engine: its classical lane may instantiate universe parameters at Prop and pass that specialized target to acceptance [2]. An inhabitant of a specialization must be labeled as such; it is not automatically an inhabitant of the original universe-polymorphic request. This report does not claim that a kernel check of the specialized term is invalid. It recommends making the distinction visible in the result algebra and receipt.

### 11.2 Proposed interfaces

The following is an architectural schema, not compiled Lean source:

```
FrozenQuery:
  target, contract, universePolicy, axiomProfile
  environmentFingerprint, grammarFingerprint, publicationObjective

Branch:
  savedLeanState, goals, dependentHoleTelescope
  residualGraph, pendingProofTasks, boundaryConstraints
  logicalCost, stableBranchId, proposalProvenance

Proposal:
  decisionKind, boundedSyntax, parentGrammar
  sourceManifest, validationEvidence

ObservationResult:
  proved(proof, dependencyReadSet)
  refuted(branchCertificate, dependencyReadSet)
  unknown(residual, blockers, reason)

PublishedResult:
  checkedProgram, checkedContractProof, exactRequestedType
  specializationLabel, axiomClosure, helperClosure
  frozenRank, searchCheckpoint, optimalityStatus
```

*A closed candidate refutation* and an *open branch refutation* should be separate constructors. The

first may be a proved counterexample to one candidate. The second needs the completion-quantified statement in (1). This prevents a common accidental promotion of a failed guess into a claim about every future refinement.

Similarly, a proposal is not an accepted search state. It must first satisfy scope, type, effect, grammar, and budget restrictions. A carrier proposal creates ordinary seed, step, and finisher holes; a model-proposed helper must be synthesized or supplied with a checked body, not added as an unexplained constant.

### 11.3 A deterministic control loop

```
freeze query, environment, grammar, objective, and proposal manifest
initialize a persistent frontier and a non-rollback work ledger
repeat until a logical bound is complete or cancellation occurs:
  choose the next deterministic scheduler checkpoint
  select a branch or proof continuation by recorded priority
  restore its Lean state
  validate caches and wake affected residual observations
  run cheap normalization and certified refutation
  if refuted: record evidence and discard this branch
  else if a complete candidate is available:
    realize recursive bodies and helpers, if needed
    check the original target, contract, and axiom profile
    update typed candidate identities and publication ranking
  else:
    compute conservative dependency components
    generate bounded native refinements and validated proposals
    add their saved successor states to the persistent frontier
  record completed work; never refund it on rollback
publish the best certified results and the exact completion status
```

This loop has no claim of universal completeness. Its bounded completeness is parameterized by the recorded grammar, the fairness of refinement enumeration, and the validity and completeness properties of the validators actually used. A heuristic front end can run within it without corrupting the meaning of a certified result.

### 11.4 Negative-result algebra

Four different negative-looking outcomes deserve distinct messages. *Contract impossible* requires a proof that no term of the requested type satisfies the contract. *Type uninhabited* requires a proof of its negation under the recorded context and profile. *Bounded grammar exhausted* requires complete resolution of the stated finite search universe. *No result before cancellation* requires no mathematical conclusion at all and must not be phrased as impossibility.

An unproved candidate that passes examples is another useful outcome, but it belongs in a separate exploratory channel. It must not be mixed into the kernel-certified candidate list with an ambiguous “success” label. The same separation applies to a program that is mathematically certified but whose executable binding has not yet been verified.

## 12 Implementation sequence and falsifiable evaluation

### 12.1 A dependency-aware sequence

Do not wait for a general carrier-invention solution before improving all other parts of the engine. Several high-value changes are independently testable.

| Stage | Deliverable                                                                  | Acceptance criterion                                                                                             |
|-------|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| P0    | Frozen request identity, typed candidate keys, exact outcome statuses        | Existing accepted results recheck; specialization and cancellation are never mislabeled                          |
| P1    | Per-observation dependencies and rollback-safe caches                        | Cached and uncached decisions agree on the same saved states; sibling-branch and delayed-assignment tests pass   |
| P2    | Persistent search frontier, deterministic scheduler, conservative components | Same logical checkpoint replays identically; no solved baseline case is lost at sufficient matched work          |
| P3    | Native motive adapter, local invariant proofs, equation-based evaluation     | Indexed map and supplied-carrier cases work without a replacement elaborator                                     |
| P4    | Bounded joint carrier synthesis and finite-state refutations                 | Known witnesses are preserved; bounded exhaustion is reported only after all candidates are resolved             |
| P5    | Capability bodies and progressively stronger angelic constraints             | Every published recursive program has checked realization and contract evidence; fuel exhaustion remains unknown |
| P6    | Dictionary-aware retrieval and optional learned proposals                    | Provider leakage is controlled; proposal and retrieval overhead is charged; replay requires no model or network  |

These are dependency stages, not promised delivery dates. P1 can begin with the current evaluator; P3 can begin with a supplied carrier. The optional learned layer should arrive after the deterministic proposal interface exists, not become the reason that basic carrier and motive testing is deferred.

### 12.2 The regression matrix

A useful test suite should combine ordinary examples with deliberately adversarial semantic cases.

**Search state and caching.** Include goals sharing a type metavariable, an index metavariable, a universe constraint, and a dependency hidden in a local declaration. Include a tactic that assigns a variable outside the apparent first goal. Test sibling branches that reuse the same local version counter with different assignments. Compare persistent-frontier and depth-first execution after sufficient exhaustive work, not merely after the same wall-clock interval.

**Carriers and recursors.** Include reverse both with append and with a continuation, optional maximum, min/max pairs, left folds with a non-associative operator, Fibonacci by tupling,

indexed vector map, an accumulating dependent function, and tree flattening with and without append as a provider. Every benchmark should state the permitted grammar and whether its reference witness lies within that grammar. An absent provider is not evidence that a carrier is impossible.

**Evaluation and proof.** Test correct and deliberately unlawful BEq instances, partial terms whose guards depend on holes, repeated recursive oracle calls, contradictory observations, well-founded definitions requiring equations, and dependent positions unsupported by a fast evaluator. For universal properties, include one family whose invariant is supplied and another where it must be invented. This separates proof-template execution from invariant discovery.

**Ranking and retrieval.** Include two expressions with the same pretty-printed display but different hidden parameters, two candidates agreeing on current probes but differing on an additional argument, legitimate constant functions, and components that are useful only as intermediates. Include local typeclass instances that change between contexts. Ensure target aliases and leaked reference bodies cannot masquerade as synthesis achievements.

### 12.3 Metrics and ablations

Report representation coverage, first certified result, best frozen-objective result, bounded-exhaustion rate, and unresolved/cancelled tasks separately. Measure wall time as a practical outcome, but compare algorithmic policies also at matched logical work. Track peak frontier memory: the proposal's cheap snapshot time does not establish cheap retention of many live states.

For evaluation, count residual nodes reused, invalidated, and recomputed; proof certificate construction and checking; and unsupported-fragment fallbacks. For proof debt, count productive wakes and repeated attempts with unchanged inputs. For retrieval, report candidate providers admitted, dictionary-solving work, and the fraction of concrete type failures attributable to each abstract feature. For carrier synthesis, record both the proposed carrier rank and the rank of the first complete certified implementation under it.

Essential ablations are cache off/on; depth-first versus persistent frontier; zero versus proved versus learned heuristic; carrier supplied versus invented; first-order versus higher-order observations; native evaluation versus proof-producing equations; and model off/on under the same total budget. Changes that improve an aggregate solved count while creating incorrect hard pruning should fail acceptance immediately.

### 12.4 What remains genuinely empirical

This analysis does not settle the best weights of a carrier grammar, the most useful abstraction depth for Mathlib retrieval, the best proof-debt scheduler, or current model accuracy on the exact requested task. Those quantities require experiments on a pinned corpus and implementation. The report instead provides well-defined experiments and interfaces that make results interpretable. In particular, it replaces unqualified claims of minimality, completeness, and reproducibility with statements whose premises can be checked.

## 13 Conclusion

Most of the expert questions do not require abandoning Leant2’s Lean-native foundation. They require sharper boundaries between a proposal, an observation, a refutation, a proof, and a search-order preference. The strongest practical route is a persistent native search with conservative dependency factoring, rollback-safe observations, checked native motive construction, local invariant obligations, and a bounded joint carrier language. Existing work already supplies higher-order live examples, dictionary-aware synthesis, and proof-producing evaluation techniques that materially shorten the research path.

The hardest remaining problem is not naming a clever carrier in isolation. It is finding an economical, type-correct combination of state, transitions, finishing computation, recursive scheme, and invariant under a realistic budget. Treating that combination as a bounded, evidence-producing synthesis problem gives useful positive and negative answers without overstating what finite examples, a kernel gate, or learned guidance can establish.

## A Complete question-to-answer index

| ID   | Question focus                                                         | Page |
|------|------------------------------------------------------------------------|------|
| R1.1 | Shared metavariables, independence, and cross-branch memoization       | 6    |
| R1.2 | Admissible cost-to-close heuristic                                     | 6    |
| R1.3 | Just-in-time learning and ranking                                      | 7    |
| R1.4 | Machine-speed independence under wall-clock budgets                    | 8    |
| R2.1 | Minimal grammar carrier from finite observations                       | 8    |
| R2.2 | Polymorphic alphabet and parametricity                                 | 9    |
| R2.3 | Backward fusion, automata, and useful complete classes                 | 10   |
| R2.4 | Motive enumeration for parameterized primitive recursion               | 12   |
| R2.5 | Generalization, rippling, and lemma speculation                        | 12   |
| R3.1 | Higher-order and dependent live bidirectional evaluation               | 13   |
| R3.2 | Assignment-driven normalization and rollback                           | 14   |
| R3.3 | A fast conservative evaluator and Lean4Lean                            | 14   |
| R3.4 | Top-down angelic termination and completeness                          | 15   |
| R4.1 | Complete bounded motive classes and accumulators                       | 16   |
| R4.2 | Canonical treatment of transports, setoids, and heterogeneous equality | 17   |
| R4.3 | Definitional unfolding policy                                          | 17   |
| R4.4 | Lean motive construction as a library                                  | 18   |
| R5.1 | Capability interpreter versus realized recursion                       | 19   |
| R5.2 | Angelic execution in top-down type-directed search                     | 20   |
| R5.3 | A small recursion basis and measured coverage                          | 20   |
| R6.1 | Interleaving value and proof holes without a universal SMT encoding    | 21   |
| R6.2 | A recognizable sufficient class of inductive proofs                    | 21   |
| R6.3 | Current Lean induction automation and bounded use                      | 22   |
| R6.4 | Principled scheduling of proof debt                                    | 23   |
| R7.1 | Dependent and universe-polymorphic reachability abstraction            | 24   |
| R7.2 | Type classes in type-guided abstraction refinement                     | 24   |
| R7.3 | Data-goal premise selection and training corpora                       | 25   |
| R8.1 | A formal objective for ranking and its empirical validation            | 26   |
| R8.2 | Cheap meaningful equivalence for dependent candidates                  | 27   |
| R9.1 | Accuracy of learned carrier/helper proposals                           | 27   |
| R9.2 | Deterministic and auditable model contribution                         | 28   |

## B Executable semantic checks and their limits

The script `semantic_checks.py` uses only Python’s standard library. Its recorded output is `semantic_results.json`. These checks are small semantic experiments, not execution of Lean or Leant2. They neither establish performance improvements nor replace the mathematical arguments in the report.

| Check                                      | Scope                                                                            | Observed result                                                      |
|--------------------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Reverse via ordinary fold and continuation | All 3,280 lists over $\{0, 1, 2\}$ of lengths zero through seven                 | Both match reversal                                                  |
| Optional min/max state                     | The same 3,280 lists, including empty input                                      | All match reference                                                  |
| Fibonacci pair update                      | Indices zero through sixty; reference uses fast doubling                         | All 61 match                                                         |
| Unary finite-state synthesis               | Boolean outputs for lengths zero through eight; target length divisible by three | Zero consistent 1- or 2-state machines; two labeled 3-state machines |
| Residual-state lower bound                 | Three tails and three prefix continuations for the unary target                  | An incompatibility clique of size three                              |
| Incomplete observation rows                | Two different partial rows with no conflicting common column                     | Rows are compatible                                                  |
| Observational congruence                   | Two functions equal on probes zero and one                                       | Application at two distinguishes them                                |
| Rollback cache identity                    | Two sibling assignments with reused version number                               | Naive key fails; qualified key separates them                        |
| Goal-count lower bound                     | Abstract two-obligation shared-assignment example                                | Remaining cost one is below goal count two                           |

For the finite-state experiment, the initial state is fixed to zero. With  $k$  states, a unary transition table has  $k^k$  possibilities and a Boolean output table has  $2^k$ . The exhaustive enumeration therefore checks 2, 16, and 216 machines for  $k = 1, 2, 3$ . The two successful three-state tables differ by state labeling. The clique witnesses are tail lengths 0, 1, 2, tested after prefix lengths 0, 1, 2; their fully known output rows are pairwise incompatible. This supplies a genuine finite-state lower bound rather than a count of syntactically distinct partial rows.

The reverse checks use Python list operations and are *not* evidence for Lean runtime complexity. The asymptotic discussion in R2 is about the stated linked-list model. The cache and heuristic examples exercise the logical countermodels; they are not demonstrations of a confirmed bug in the current Leant2 implementation.

## C Evidence status and reproducibility

The primary research sources and official Lean documentation were consulted on 21 September 2026. Repository analysis is pinned to the SHA on the title page; native API observations are pinned to Lean 4.34. The proposal’s older benchmark measurements are cited as historical reports. The mathematical arguments in this report are written proofs, not Lean-formalized theorems.

No Lean executable was available in the working environment, so none of the proposed Lean interfaces or examples was compiled here. The report does not claim a Leant2 patch, a rerun of its acceptance harnesses, cross-machine replay measurements, or model accuracy data. The Python semantic checks were executed successfully, and the accompanying PDF was compiled from this source.

The inspected material did not establish a precise current sauto unfolding policy or a published accuracy measurement matching R9.1’s exact carrier-proposal task. The relevant sections give explicit recommendations and evaluation procedures without filling those evidence gaps with invented results. Claims of absence from the literature are deliberately avoided; “not established by the sources investigated” is narrower and more accurate.

To rebuild, run `pdflatex leant2_expert_answers.tex` twice, or run `latexmk -pdf leant2_expert_answers.tex`. A normal TeX Live installation with the packages named in the preamble is sufficient. Run `python semantic_checks.py` to reproduce the semantic result file.

## References

- [1] V. Reshetnikov. *Leant2: Further Improvements*, question-bearing sections 05–12 and expert summary 15. Repository snapshot 784664ff34e819422510a4d470ab07fe48bb736b, inspected 21 September 2026. <https://github.com/VladimirReshetnikov/Leant2/tree/784664ff34e819422510a4d470ab07fe48bb736b/docs/proposals/11-further-improvements>.
- [2] V. Reshetnikov. *Leant2 implementation*: Leant2/Engine.lean, Leant2/Native/Transaction.lean, Leant2/Accept/Gate.lean, and lean-toolchain. Same pinned snapshot as [1]. Engine source; transaction source; gate source.
- [3] V. Reshetnikov. *Leant2 Further Improvements: Evidence*, sections/01-evidence.tex. Same pinned snapshot as [1]; the acceptance table itself reports commit 33cec8b. Pinned evidence file.
- [4] J. Limperg and A. H. From. An Extensible Theorem Prover for Lean. *CPP*, 2023. DOI: 10.1145/3573105.3575671. <https://people.compute.dtu.dk/ahfrom/aesop-camera-ready.pdf>.
- [5] C. Norman and J. Avigad. Canonical for Automated Theorem Proving in Lean. 2025. <https://arxiv.org/abs/2504.06239>.
- [6] C. Norman and J. Avigad. Implementing Dependent Type Theory Inhabitation and Unification. 2026. <https://arxiv.org/abs/2603.01463>.
- [7] S. Barke, H. Peleg, and N. Polikarpova. Just-in-Time Learning for Bottom-Up Enumerative Synthesis. *PACMPL*, OOPSLA, 2020. DOI: 10.1145/3428295. <https://arxiv.org/abs/2010.08663>.
- [8] J. Gibbons, G. Hutton, and T. Altenkirch. When Is a Function a Fold or an Unfold? *Electronic Notes in Theoretical Computer Science*, 44(1), 2001. <https://www.cs.ox.ac.uk/publications/publication/2368-abstract.html>.
- [9] T. Weber and J. Caldwell. Constructively Characterizing Fold and Unfold. *LOPSTR 2003, Revised Selected Papers*, LNCS 3018, 2004. DOI: 10.1007/978-3-540-25938-1\_11. [https://www.cs.uwo.edu/~jlc/papers/fold\\_unfold.pdf](https://www.cs.uwo.edu/~jlc/papers/fold_unfold.pdf).
- [10] D. Angluin. Learning Regular Sets from Queries and Counterexamples. *Information and Computation*, 75:87–106, 1987. DOI: 10.1016/0890-5401(87)90052-6.
- [11] J.-P. Bernardy, P. Jansson, and K. Claessen. Testing Polymorphic Properties. *ESOP*, LNCS 6012, pp. 125–144, 2010. DOI: 10.1007/978-3-642-11957-6\_8. [https://publications.lib.chalmers.se/records/fulltext/local\\_99387.pdf](https://publications.lib.chalmers.se/records/fulltext/local_99387.pdf).
- [12] K. Claessen, M. Johansson, D. Rosén, and N. Smallbone. Automating Inductive Proofs Using Theory Exploration. *CADE-24*, LNCS 7898, pp. 392–406, 2013. DOI: 10.1007/978-3-642-38574-2\_27. <https://www.cse.chalmers.se/~jomoa/papers/hipspect-ade.pdf>.
- [13] J. Lubin, N. Collins, C. Omar, and R. Chugh. Program Sketching with Live Bidirectional Evaluation. *PACMPL*, 4(ICFP), article 109, 2020. DOI: 10.1145/3408991. Extended version: <https://arxiv.org/abs/1911.00583>.
- [14] M. A. Hammer, K. Y. Phang, M. Hicks, and J. S. Foster. Adapton: Composable, Demand-Driven Incremental Computation. *PLDI*, 2014. DOI: 10.1145/2594291.2594324. <https://www.cs.umd.edu/~mwh/papers/hammer13adapton.html>.
- [15] Lean FRO and contributors. *The Lean Language Reference*, version 4.34.0, Tactic Reference: `cbv`, `decide_cbv`, and their limitations. <https://lean-lang.org/doc/reference/4.34.0/Tactic-Proofs/Tactic-Reference/>.
- [16] Lean FRO and contributors. *The Lean Language Reference*: Recursive Definitions. Online reference consulted 21 September 2026. <https://lean-lang.org/doc/reference/latest/Definitions/Recursive-Definitions/>.

- [17] M. Carneiro. Lean4Lean: Towards a Verified Typechecker for Lean, in Lean. *TYPES 2025*, LIPIcs 384, article 2, published 30 July 2026. DOI: 10.4230/LIPIcs.TYPES.2025.2. <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2025.2>.
- [18] A. Miltner, A. T. Nuñez, A. Brendel, S. Chaudhuri, and I. Dillig. Bottom-Up Synthesis of Recursive Functional Programs Using Angelic Execution. *PACMPL*, 6(POPL), article 21, 2022. DOI: 10.1145/3498682. <https://arxiv.org/abs/2107.06253>.
- [19] Ł. Czajka. Practical Proof Search for Coq by Type Inhabitation. *IJCAR*, pp. 28–57, 2020. DOI: 10.1007/978-3-030-51054-1\_3. Current implementation-level unfolding policy was not verified in this report.
- [20] Lean contributors. `src/Lean/Meta/Tactic/Induction.lean`, version v4.34.0; native induction, index extraction, and recursor-prefix APIs. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Induction.lean>.
- [21] N. Polikarpova, I. Kuraj, and A. Solar-Lezama. Program Synthesis from Polymorphic Refinement Types. *PLDI*, 2016. DOI: 10.1145/2908080.2908093. <https://cseweb.ucsd.edu/~npolikarpova/publications/pldi16.pdf>.
- [22] T. Zhu, J. Clune, J. Avigad, A. Q. Jiang, and S. Welleck. Premise Selection for a Lean Hammer. *ICLR*, 2026; preprint first posted in 2025. <https://arxiv.org/abs/2506.07477>.
- [23] Z. Guo, M. James, D. Justo, J. Zhou, Z. Wang, R. Jhala, and N. Polikarpova. Program Synthesis by Type-Guided Abstraction Refinement. *PACMPL*, 4(POPL), article 12, 2020. DOI: 10.1145/3371080. [https://ranjitjhala.github.io/static/hoogle\\_plus.pdf](https://ranjitjhala.github.io/static/hoogle_plus.pdf).
- [24] Lean contributors. `src/Lean/LibrarySuggestions/Basic.lean`, version v4.34.0; selector and configuration API. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/LibrarySuggestions/Basic.lean>.
- [25] Premise-selection project contributors. *premise-selection README*, consulted 21 September 2026; cloud selector interface, caching, and cold-start behavior. Inspected README blob: ba823550133d3b56a756787eab334f8151e03808. <https://github.com/hanwenzhu/premise-selection/blob/main/README.md>.
- [26] N. Natarajan, D. Simmons, N. Datha, P. Jain, and S. Gulwani. Learning Natural Programs from a Few Examples in Real-Time. *AISTATS*, PMLR 89:1714–1722, 2019. <https://proceedings.mlr.press/v89/natarajan19a.html>.
- [27] M. Mayer et al. User Interaction Models for Disambiguation in Programming by Example. *UIST*, 2015. DOI: 10.1145/2807442.2807459. <https://www.microsoft.com/en-us/research/publication/user-interaction-models-disambiguation-programming-example/>.